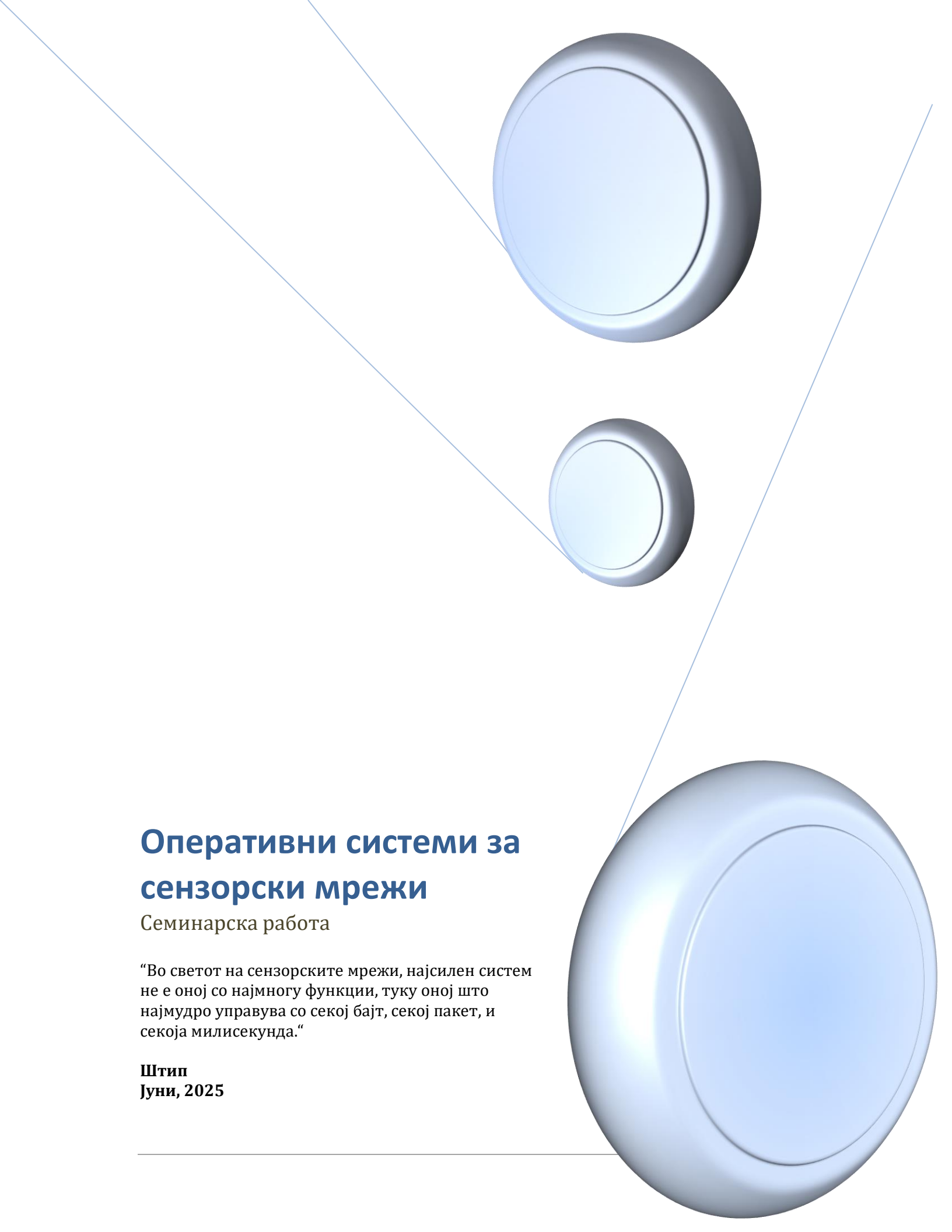


An abstract graphic featuring three blue, 3D-rendered spheres of varying sizes. Two thin blue lines originate from the top left and extend diagonally across the page, passing behind the spheres. The largest sphere is at the bottom right, a medium-sized one is at the top right, and a smaller one is in the center. The background is a light, neutral color.

Оперативни системи за сензорски мрежи

Семинарска работа

“Во светот на сензорските мрежи, најсилен систем не е оној со најмногу функции, туку оној што најмудро управува со секој бајт, секој пакет, и секоја милисекунда.”

Штип
Јуни, 2025

УНИВЕРЗИТЕТ „ГОЦЕ ДЕЛЧЕВ”- ШТИП
ФАКУЛТЕТ ЗА ИНФОРМАТИКА
Компјутерско инженерство и технологии

СЕМИНАРСКА РАБОТА
Оперативни системи за сензорски мрежи



Ментор,
Проф. Д-р Зоран Златев

Студент,
Мерсие Адемова
број на индекс:102856

Штип, јуни 2025

Содржина

1. Вовед.....	3
Сензори	4
Микроконтролер	6
2. Сензорски мрежи.....	7
Дефиниција.....	7
Архитектура.....	9
Класификација.....	10
Топологија.....	12
3. Примена на сензорски мрежи	13
Животна средина (Environmental Monitoring).....	13
Прецизна земјоделска дејност (Precision Agriculture)	13
Индустриска примена	13
Медицина и здравство	14
Сигурност и заштита.....	14
Паметни градови (Smart Cities).....	14
Следење на катастрофи и итни состојби.....	14
4. Основи на оперативни системи за сензорски мрежи	15
5. Историја и развој	16
TinyOS (2000)	16
Contiki OS (2002)	16
RIOT OS (2013).....	17
MANTIS OS (2005)	17
LiteOS (2008)	17
6. Класификација.....	17
Според архитектурата на извршување	17
Според програмскиот модел	18
Според поддршката за комуникациски протоколи	18
Според поддршката за реално време (Real-time)	18
Според целната примена	19
7. Управување со ресурси	19
Управување со меморија.....	19

Управување со енергија.....	20
Управување со процеси	21
Управување со комуникациски ресурси	21
Управување со складирање (storage management)	22
8. Мрежна комуникација.....	22
Протоколи за комуникација	23
Управување со мрежен сообраќај.....	24
Предизвици	24
9. Перформанси и функционалности	24
Потрошувачка на енергија	25
Поддршка за мултитаскинг	26
10. Процеси и сервиси	26
Модел на процеси.....	26
Системски сервиси	27
Управување со процеси	28
11. TinyOS.....	28
12. Contiki OS	30
13. RIOT OS.....	32
14. MANTIS OS	34
15. LiteOS	35
16. Критична анализа.....	37
Критички осврт.....	38
17. Заклучок	39
18. Користена литература	40

Оперативни системи за сензорски мрежи

1. Вовед во сензорски мрежи

1. Вовед

Постојаното намалување на цените на технологиите за безжична комуникација, како и на сензорите и уредите за процесирање, доведе до појавување на нова генерација паметни системи кои максимално ја искористуваат моќта на работењето во “заедница”. Ниската цена овозможи комбинирање на овие технологии во уреди со мали димензии, па започнаа да се имплементираат системи со користење на десетици, па дури и неколку илјади вакви уреди. Со вградувањето на истите во самоорганизирачки мрежи им овозможуваат да функционираат како еден хомоген уред, започна конструирањето на еден нов вид ИКТ- системи познати како безжични сензорски мрежи.

Предвидувањата и очекувањата се дека безжичните сензорски мрежи (WSN) ќе имаат поголемо влијание врз сите аспекти на начинот на живеење денес, притоа носејќи широк спектар на подобрување на технологиите во кои сега се применуваат, како што се системи за надгледување, медицинските апликации, воените системи и сл. Потенцијалот кој го имаат безжичните сензорски мрежи, претставува и своевидна гаранција дека примената на нивната технологија ќе се прошири и ќе навлезе и во други области, како што се автомобилската индустрија, земјоделството, транспортот, управувањето со енергетски системи или секаде каде што има потреба од дистрибуирано мерење и контрола на голем број параметри.

Оперативните системи (ОС) за сензорски мрежи претставуваат посебна категорија на софтверска платформа дизајнирана да работи во ограничени

ресурси (батерија, меморија, процесорска моќ) и специфични услови на безжични сензорски мрежи (WSN).

Сензорските мрежи се клучна технологија за современи апликации во различни домени како што се индустријата, здравството, земјоделството и паметни градови. Овие мрежи бараат специфични мрежни системи кои ќе овозможат ефикасно управување со ограничени ресурси и реално-временски барања.

Оваа семинарска работа има за цел да ги анализира основните концепти, архитектурата, примената и предизвиците поврзани со оперативните системи за сензорски мрежи.

Уште на самиот почеток ќе ги разгледаме клучните поими кои ќе ни бидат потребни за полесна и поедноставна понатамошна анализа:

Сензори

Сензор или **детектор** е било која направа што мери една физичка големина и ја претвора во облик погоден за понатамошна обработка, односно за читање од страна на набљудувачот или од инструментот (најчесто електронски). Излезната величина од мерниот инструмент се бара да биде правопрпорционална со неговата влезна величина. Делот од мерниот инструмент кој непосредно чувствува некои промени во процесот се нарекува сетилен елемент.

Во однос на принципот на работа, мерните инструменти може да се поделат на отпорнички, капацитивни, индуктивни, електромагнетни, пиезоелектрични, механички, хидраулични, пневматски и др. Електричната и механичката величина се најсоодветни за мерење па тие најчесто се јавуваат како излезни величини, па сензорите врз основа на излезната величина се поделени на **електрични** и **механички**.



Електричните сензори работат врз принцип на познатите електрични ефекти. На влезот од сензорот може да биде користена енергија во која било форма, додека на неговиот излез се јавува електрична величина која одговара на влезната величина.

Основните критериуми за оценка на квалитетот на мерните инструменти се:

- *Мерното подрачје*, кое ги опфаќа вредностите на мерената и процесната величина за која сензорот може да се употреби. Се изразува преку најголемата и најмалата вредност на мерената величина;
- *Мерниот опсег* мора да го покрива опсегот на промените на мерената процесна величина и претставува разлика на вредностите на оваа величина на горната и долната граница на мерното подрачје;
- *Мерниот сигнал* е непрекината променлива величина, по форма аналогна на промените на мерната процесна величина. Мерниот сигнал претставува излез на сензорот. Во последно време се појавуваат и дигитални сензори со

дискретен мерен сигнал, кои вредности на мерната процесна величина ја даваат во бројчана форма;

- *Статичката грешка* е отстапување од вистинската вредност на мерената процесна величина кога влезната величина е константна;
- *Динамичката грешка* е грешка која настанува поради инертноста на подвижните делови на сензорот, па покажаната вредност не може да ја следи вистинската вредност на мерената процесна величина;
- *Додатна грешка* се појавува поради промената на надворешните услови кои се јавуваат поради менување на температурата, притисокот и сл.;
- *Статичката карактеристика* на сензорот е определена со зависноста на влезната величина Y од промените на влезната величина X : $Y = f(X)$.

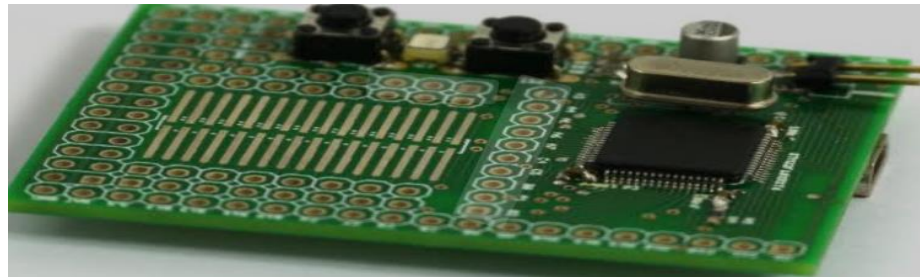
Микроконтролер

Микроконтролерот е интегрално коло кое работи со процесор кој исто така се нарекува процесор и мемориски единици ROM и RAM. Сите овие делови меѓусебно се поврзани и работат едни со други во рамките на микроконтролерот. Микроконтролерите се особено корисни во апликации кои бараат автоматизација, контрола и мониторинг, како што се домашни апарати, системи за контрола на моторот, вградени системи, медицински уреди, безбедносни системи, електронски играчки и широк спектар на електронски производи.

Компоненти на микроконтролерот се:

- *Процесор (процесор)* одговорен за извршување на инструкции и пресметки;
- *Меморија* која вклучува меморија само за читање (ROM) за складирање на програмата и меморија за случаен пристап (RAM) за привремени податоци;

- *Влезно/излезни периферни уреди (I/O)* кои му овозможуваат на микроконтролерот да комуницира со други компоненти, како што се сензори и актуатори;
- *Тајмери и бројачи* кои помагаат во извршувањето на задачите во одредени временски интервали;
- *Конвертори* на A/D и D/A за конвертирање на аналогните сигнали во дигитални и на дигиталните во аналогни.

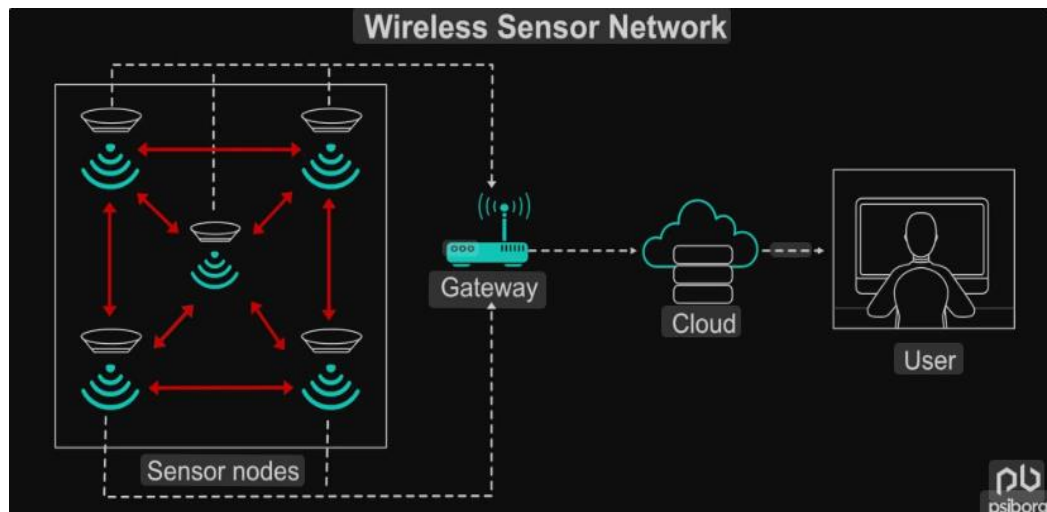


2. Сензорски мрежи

Дефиниција

Сензорската мрежа е структура која се состои од сензори, пресметковни единици и комуникациски елементи со цел снимање, набљудување и реагирање на настан или феномен. Настаните може да бидат поврзани со било што како физичкиот свет, индустриска средина, биолошки систем или рамка за ИТ (информациска технологија), додека телото за контрола или набљудување може да биде апликација за потрошувачи, влада, цивилен, воен или индустриски ентитет. Таквите сензорски мрежи може да се користат за далечинско мерење, медицинска телеметрија, надзор, мониторинг, собирање на податоци и сл. Типичната сензорска мрежа се состои од сензори, контролер и комуникациски систем. Ако комуникацискиот систем во сензорска мрежа се имплементира со помош на безжичен протокол, тогаш мрежите се познати како безжични сензорски мрежи (WSN). Според технолозите и истражувачите, безжичните сензорски мрежи како ентитет се важна технологија за дваесет и првиот век.

Безжичната сензорска мрежа (wireless sensor network, WSN) се состои од просторно распределени автономни сензори или детектори за следење на физичките услови на околината, како што се температура, звук, притисок итн. и кооперативно ги пренесуваат своите податоци преку мрежата до главното место. Посовремените мрежи се директно двонасочени, и исто така овозможуваат контрола над активноста на сензорот.



Елементи на типичната безжична сензорска мрежа се сензорскиот јазол и мрежата. Сензорскиот јазол во WSN се состои од четири основни компоненти: напојување, единица за процесирање, комуникациски систем и конвертор. Главната процесорска единица, која обично е микропроцесор или микроконтролер, врши интелигентна обработка и манипулација на податоците, кои претходно се соберени од страна на сензорот во вид на аналогни податоци од физичкиот свет и конвертирани во дигитални податоци. Системот за комуникација се состои од радио систем, обично радио со краток домен за пренос и прием на податоци. Бидејќи сите компоненти се уреди со мала моќност, мала батерија како CR-2032, се користи за напојување на целиот систем. Сензорскиот јазол содржи и други важни карактеристики како обработка, комуникација и единица за складирање. Со сите овие карактеристики, компоненти и подобрувања, сензорскиот јазол е одговорен за собирање на податоци од физичкиот свет,

мрежна анализа, корелација на податоци и спојување на податоците од друг сензор со сопствените податоци.

Архитектура

Кога голем број на сензорски јазли се распоредени во голема област за кооперативно следење на физичката средина, вмрежувањето на овие сензорски јазли е подеднакво важно. Сензорскиот јазол во WSN не само што комуницира со други сензорски јазли туку и со базната станица користејќи безжична комуникација. Базната станица испраќа команди до јазлите на сензорот и сензорните јазли ја извршуваат задачата соработувајќи меѓу себе. По собирањето на потребните податоци, сензорните јазли ги испраќаат податоците назад до базната станица. Базната станица, исто така делува како порта за други мрежи преку интернет. По добивањето на податоците од сензорните јазли, базната станица врши едноставна обработка на податоците и ги испраќа ажурираните информации до корисникот користејќи интернет. Иако преносот на долги растојанија евозможен, потрошувачката на енергија за комуникација ќе биде значително повисока од собирањето и пресметувањето на податоци. Оттука, вообичаено се користи мрежна архитектура Multi-hop. Наместо една единствена врска помеѓу сензорниот јазол и базната станица, податоците се пренесуваат преку еден или повеќе средни јазли. Ова може да се имплементира на два начини: архитектура на рамна мрежа и хиерархиска мрежна архитектура.

Во рамна архитектура, базната станица испраќа команди до сите сензорски јазли, но сензорниот јазол со соодветно барање ќе одговори користејќи ги неговите врсници преку патека со повеќе скокови.

Во хиерархиската архитектура, група на сензорски јазли се формираат како кластер и сензорските јазли пренесуваат податоци до соодветните глави на кластерите. Главите на кластерот потоа можат да ги пренесат податоците до базната станица. Оттука, карактеристиките на една WSN ќе се разликуваат од оној на друг WSN.

Класификација

Без оглед на апликацијата, безжичните сензорски мрежи може да се класифицираат во следниве категории:

- Еднохоп и мултихоп мрежи:
 - Еднохоп (single-hop) - сите сензорски јазли директно комуницираат со една базна станица, која е позиционирана во близина. Овој модел е едноставен, но ограничен поради растојанието.
 - Мултихоп (multi-hop) – податоците се пренесуваат преку други сензорски јазли (репетитори) до базната станица. Овој пристап овозможува покривање на поголеми подрачја и зголемување на скалабилноста.
- Статички и мобилни мрежи:
 - Статички мрежи – сите јазли имаат фиксна позиција по распоредувањето. Најчесто се користат во околина каде што мобилноста не е потребна, како што се земјоделски или индустриски апликации.
 - Мобилни мрежи – сензорските јазли или базните станици можат да се движат. Овие мрежи се користат во апликации како следење на животни, воени мисии или биолошки истражувања.
- Хомогени и хетерогени мрежи:

- Хомогени мрежи – сите јазли имаат иста хардверска и софтверска конфигурација.
- Хетерогени мрежи – постојат различни типови на јазли, на пример, некои се специјализирани за обработка, некои за складирање или комуникација, што овозможува оптимизација на ресурсите.
- Реконфигурирачки и не-само-конфигурирачки мрежи:
 - Реконфигурирачки (self- configurable) – јазлите можат автоматски да ја адаптираат својата конфигурација според условите во околината.
 - Не-само-конфигурирачки – потребна е човечка интервенција или надворешен систем за промена на конфигурацијата.
- Детерминистички и недетерминистички мрежи:
 - Детерминистички – позицијата на секој јазол се планира однапред, што овозможува предвидлива покриеност и поврзаност.
 - Недетерминистички – јазлите се распоредуваат случајно, на пример, со фрлање од воздух или во непристапна територија. Ова бара динамички механизми за организирање и самоконфигурација.

Да сумираме: сензорската мрежа е составена од неколку стотици до неколку илјадници “јазли” каде што еден јазол е поврзан со еден или повеќе сензори. Секој таков јазол на сензорската мрежа е составен од неколку делови: приемник со внатрешна антена или поврзување со надворешна антена, микроконтролер, електронски кола за поврзување на сензорите и извор на енергија, обично батерија или вградена форма за собирање на енергијата. Сензорските јазли исто така може да варираат во однос на големината, може да се колку кутија за чевли или колку зрно прашина, иако тоа “зрно” во вистински

микроскопски димензии допрва треба да биде создадено. Цената на сензорските јазли е исто така променлива, таа се движи од неколку до стотици долари, во зависност од комплексноста на поединечните сензорски јазли.

Топологија

Промените и одржувањето на топологијата може да биде разгледано во три фази, односно фаза на распоредување, фаза на пост-распоредување и фаза на повторно распоредување.

Почетната топологија се поставува за време на распоредувањето. Јазлите може да бидат распоредени еден по друг, или може да бидат распоредени масивно, на пример, испуштајќи ги еден по еден од авион.

Промените во топологијата за време на фазата на пост-распоредување се должат на неуспесите на јазолот и промените на позицијата на јазлите поради мобилноста. За време на фазата на повторно позиционирање, се распоредуваат дополнителни јазли во мрежата. Ова може да се случи во секое време.

Топологијата на сензорските мрежи може да варира од едноставна ѕвездаста мрежа до напредна мулти-хоп безжична решеткава мрежа. Техниките на пропација ,може да се случат со помош на насочување или поплавување.

3. Примена на сензорски мрежи

Примената на сензорски мрежи е широка и се проширува постојано поради напредокот во микроелектрониката, безжичната комуникација и вградените системи. Во продолжение се дадени главните области каде што се користат сензорските мрежи.

Животна средина (Environmental Monitoring)

- Следење на квалитетот на воздухот и водата: *мерење на РМ честички, NO_x , CO_2 , pH, температура на вода;*
- Метеоролошко следење: *температура, влажност, атмосферски притисок, врнежи;*
- Детекција на шумски пожари: *ран сигнал преку температура и чад;*
- Следење на живи суштества и растенија: *следење на миграција, популација или состојба на шумски екосистем.*

Прецизна земјоделска дејност (Precision Agriculture)

- Мерење на влажност на почва. Температура, pH
- Оптимизација на наводнување и губрење
- Автоматизирани алатки за подобрување на принос и намалување на ресурси
- Следење на штетници или болести кај култури

Индустриска примена

- Следење на производни процеси и опрема: *детекција на вибрации, притисок, температура;*
- Предиктивна и превентивна одржливост (maintenance): *известување за можни дефекти;*
- Контрола на складишта и логистика: *следење на услови (температура, влажност) за складирање на храна или лекови.*

Медицина и здравство

- Телемедицина и здравствен надзор на пациенти: *следење на срцев ритам, температура, ниво на гликоза;*
- Сензори вградени во облека или уреди: *помагаат во следење на хронични болести.*

Сигурност и заштита

- Воени апликации: следење на подрачја, детекција на движење, присуство на непријатели;
- Граници и безбедносен надзор: сензори за движење, звук или вибрации;
- Инфраструктурна безбедност: следење на мостови, брани, тунели (деформации, вибрации).

Паметни градови (Smart Cities)

- Управување со сообраќајот: следење на проток на возила, паметни семафори;
- Управување со отпад: сензори кои известуваат кога контејнер е полн;
- Енергетска ефикасност: следење и управување со потрошувачката на струја.

Следење на катастрофи и итни состојби

- Детекција на земјотреси, свлачишта, поплави;
- Сензори на критични локации за предупредување и рана интервенција.

Основни карактеристики на оперативните системи за сензорски мрежи

4. Основи на оперативни системи за сензорски мрежи

Оперативните системи овозможуваат управување со ресурси, меморија, процеси и комуникација. За разлика од класичните ОС, овие се оптимизирани за ниска потрошувачка и специфични задачи.

Оперативниот систем (OS) во сензорски јазол е лесна, енергетски ефикасна платформа која:

- Управува со внес и излез од сензорите;
- Го контролира комуникацискиот модул;
- Поддржува мултитаскинг;
- Обезбедува мрежна комуникација и протоколи;
- Обезбедува интерфејс за развој на апликации.

Главните карактеристики на ваквите оперативни системи се:

- Мала мемориска трага: оперативниот систем мора да работи со неколку килобајти RAM/ROM;
- Енергетска ефикасност: сите процеси мора да се оптимизираат за минимална потрошувачка на енергија;
- Реално време: мора да поддржува одговор во реално време за критични апликации;
- Поддршка за мрежи: вклучува протоколи за безжична комуникација и мрежна топологија;
- Поддршка за настани: претежно користи event-driven модел наместо традиционален multitasking.

Најпознати оперативни системи за сензорски мрежи се:

- TinyOS;
- Contiki OS;
- RIOT OS;
- MANTIS OS;
- LiteOS;

5. Историја и развој

Развојот на безжичната сензорска мрежа е инспирирана од воени примени како што е надзорот на воено поле, а денес такви мрежи се користат во многу индустриски и потрошувачки примени, како што се индустриските процеси за следење и контрола на машините, за медицински надзор итн.

TinyOS (2000)

TinyOS е проект од Универзитетот Беркли (UC Berkeley). Тој е првиот специјализиран оперативен систем за безжични сензорски мрежи. Развиен е како дел од програмата за смарт-дистрибуирани системи и е напишан на нов јазик - nesC. Благодарение на неговата ефикасност и минимален ресурсен отпечаток брзо станал стандард во академијата. До сега се користел во голем број на истражувања, експерименти и реални проекти.

Contiki OS (2002)

Contiki OS е развиен од страна на Адам Данкел (Adam Dunkels) во Шведскиот институт за компјутерска наука. Неговата главна цел е да овозможи лесна интернет поврзаност (IPv6, 6LoWPAN) за уреди со ниска моќ. Тој е првиот оперативен систем кој нуди вградена поддршка за интернет на нештата (IoT). Contiki OS се развива под отворена лиценца и стана многу популарен во индустријата и образованието. Вклучува симулатор (Cooja), што го прави идеален за тестирање.

RIOT OS (2013)

RIOT OS започнал како академски проект во Германија (Freie Universität Berlin). Развиен е како реално временски оперативен систем со поддршка за повеќенишно извршување, за разлика од TinyOS и Contiki OS. RIOT OS е компатибилен со POSIX, што го прави полесен за програмирање и портирање. Тој за брзо време е прифатен од страна на заедницата за IoT и поддржува широк спектар на уреди и комуникациски протоколи.

MANTIS OS (2005)

MANTIS OS е развиен од Универзитетот во Колорадо. Неговата цел е да понуди реално мултитаскинг искуство со поддршка за нишки. Користејќи ги предностите на C вклучува вграден shell за управување.

Досега се користел во неколку академски и истражувачки проекти, но нема добиено голема популарност како TinyOS или Contiki OS.

LiteOS (2008)

LiteOS е развиен од Хуажонг Универзитетот за наука и технологија (Кина). Тој е инспириран од Linux и нуди нишки, датотечен систем и далечинско ажурирање. Поддржува хибридни апликации со можност за програмирање повеќе задачи. LiteOS главно се користи во Азија во индустриски и академски истражувања.

6. Класификација

Оперативните системи за сензорски мрежи може да бидат класифицирани според неколку критериуми кои ги одразуваат нивните архитектонски, функционални и апликативни својства.

Според архитектурата на извршување

- Event-driven (настански управувани):
Нивното извршување се базира на обработка на настани и погодни се за енергетски ефикасни системи: *TinyOS* и *Contiki OS* (делумно);
- Thread-based (врз основа на нишки):
Поддржуваат вистински мултитаскинг (многу процеси паралелно) и погодни се за покомплексни апликации, но трошат повеќе ресурси: *RIOT OS*, *MANTIS OS*, *LiteOS*.

Според програмскиот модел

- Модуларни системи:
Оперативниот систем е изграден од мали, независни компоненти, овозможувајќи лесно додавање на нови функционалности: *TinyOS*, *RIOT OS*;
- Монолитни системи:

Оперативниот систем е цврст блок со сите функции интегрирано во него, при што е тешко да се модификува или надградува: *стари верзии на LiteOS*.

Според поддршката за комуникациски протоколи

- Локални протоколи:
Поддржуваат комуникација само внатре во мрежата: *TinyOS во основна конфигурација*;
- Интернет поврзани (IoT поддршка):
Вклучуваат IPv6, 6LoWPAN, CoAP, и други IoT стандарди со овозможена директна комуникација со интернет: *Contiki OS, RIOT OS, LiteOS*.

Според поддршката за реално време (Real-time)

- Soft Real-Time:
Имаат ограничена гаранција за време на одговор и погодни се за апликации каде доцнењето не е критично: *Contiki OS, MANTIS OS*;
- Hard Real-Time:
Имаат гарантирано време на одговор за критични задачи, што е потребно кај индустриски или воени апликации: *RIOT OS, специјални варијанти на LiteOS*.

Според целната примена

- Образовни и истражувачки: *TinyOS, Contiki, RIOT*;
- Индустриски апликации: *LiteOS, RIOT*;
- Smart Home и IoT решенија: *RIOT, Contiki, LiteOS*;
- Воени и безбедносни цели: *специјални варијанти на RIOT или real-time оперативни системи*.

7. Управување со ресурси

Безжичните сензорски мрежи (WSN) функционираат во околина со екстремно ограничени ресурси – минимална енергија, мала меморија, ограничена процесорска моќ и нестабилна мрежна поврзаност. Затоа, ефикасното управување со овие ресурси е една од главните задачи на оперативните систем.

Управувањето со ресурси во сензорските оперативни системи бара прецизна контрола и оптимизација, при што најчесто се избира компромис помеѓу функционалност, стабилност и енергетска ефикасност. Начинот на кој оперативниот систем го решава овој предизвик има директно влијание врз животниот век, сигурноста и успешноста на мрежата.

Управување со меморија

Меморијата во сензорските јазли е ограничена (2-10 KB RAM), па оперативниот систем мора да нуди механизми кои овозможуваат ефикасно и безбедно користење на меморискиот простор.

- **Статичко распределување на меморија** најчесто се користи кај *TinyOS*. Меморијата се алоцира во време на компилација, што ја прави предвидлива и сигурна, но ја ограничува флексибилноста.
- **Динамичко управување со меморија** е поддржано кај *RIOT*, *Contiki* и други. Меморијата се доделува и ослободува за време на извршување, но се зголемува ризикот од фрагментација и memory leaks.
- **Распределба на стек** – стековите се строго ограничени. Во *TinyOS*, на пример, целиот систем може да користи еден заеднички стек.

- **Бафери и кеширање** – оперативниот систем мора да менаџира прием и испраќање на податоци преку ограничени бафери, користејќи кружни структури, приоритети и менаџмент на опашки.

Управување со енергија

Поради ограничената енергија (батерија), WSN оперативниот систем мора да го минимизира трошењето на енергија преку следниве техники:

- **Duty cycling:** Активирање на модулите само кога се потребни, а остатокот од времето уредот спие (sleep mode);
- **Power-aware scheduling:** Распоредување на задачи според нивниот енергетски трошок;
- **Радио контрола:** Радио модулот (најголем потрошувач на енергија) се вклучува само при комуникација;
- **Модулација на фреквенција:** Намалување на фреквенцијата на процесорот кога е можно.

Управување со процеси

Оперативниот систем мора да координира повеќе задачи, настани и прекини, и тоа:

- **Настански системи (TinyOS):** Управуваат со настани и мали задачи, без класични нишки;

- **Нитки базирани системи (RIOT):** Овозможуваат паралелно извршување со приоритети и распределување;
- **Хибридни решенија (Contiki):** Користат protothreads – комбинација од лесни нишки и настани.

Управување со комуникациски ресурси

Оперативниот систем исто така е одговорен за:

- **Координација на мрежната комуникација;**
- **Избегнување на конфликти (collisions)** преку MAC протоколи;
- **Обезбедување QoS (квалитет на сервис)** преку приоритетни пакети и обработка.

Управување со складирање (storage management)

Некои безжични сензорски мрежни уреди имаат надворешна Flash меморија, која оперативниот систем треба да ја менаџира за:

- Зачувување на податоци кога уредот е offline;
- Историја на мерења (logging);
- Управување со датотеки и префрлување на податоци

8. Мрежна комуникација

Мрежната комуникација е сржта на функционирањето на безжичните сензорски мрежи, бидејќи податоците што ги собираат сензорските јазли

треба да се пренесат во базната станица или до други јазли за понатамошна анализа. Оперативниот систем игра клучна улога во организирањето, управувањето и оптимизирањето на комуникацијата.

Мрежната комуникација во безжичните сензорски мрежи е внимателно оптимизирана од страна на оперативниот систем за да се постигне рамнотежа помеѓу енергетската ефикасност, сигурноста и брзината на преносот. Успешната комуникација е клучна за корисноста на ваквите мрежи, особено во критична апликации како што се безбедносен надзор, мониторинг на катастрофи и индустриска автоматизација.

Основни елементи за комуникација се:

- **Сензорски јазол** (sensor node): Собира и пренесува податоци;
- **Базна станица** (sink): Ги прима податоците од сензорската мрежа и ги проследува кон надворешни системи;
- **Радио интерфејс**: Обезбедува безжична поврзаност преку RF комуникација.

Додека, модели на комуникација се:

- **Peer-to-peer**: Јазлите комуницираат директно едни со други;
- **Multihop**: Податоците се пренесуваат преку неколку посредни јазли;
- **Ad-hoc**: Мрежата автоматски се организира без централизирана контрола.

Протоколи за комуникација

Безжичните сензорски мрежи користат специјализирани комуникациски протоколи кои се оптимизирани за ниска потрошувачка на енергија, минимална латентност и стабилна преносна мрежа.

➤ Физички и MAC слој:

- **IEEE 802.15.4:** Најкористен стандард за нискоенергетска комуникација; основа за ZigBee и 6LoWPAN;
- **B-MAC (Berkeley MAC):** Енергетски ефикасен, едноставен MAC протокол;
- **X-MAC:** Подобен MAC со скратени преноси за да се намали времето на радио активност.

➤ **Мрежен слој:**

- **6LoWPAN (IPv6 over Low-Power Wireless Personal Area Networks):** Овозможува користење на IPv6 адресирање во WSN;
- **RPL (Routing Protocol for Low-Power and Lossy Networks):** Протокол за рутирање оптимизиран за ограничени мрежи;
- **Collection Tree Protocol (CTP):** Протокол за собирање на податоци, често користен во TinyOS.

Управување со мрежен сообраќај

Оперативниот систем е одговорен за:

- **Бафер менаџмент:** Привремено чување на пакетите при конгестија;
- **Контрола на пристап до медиумот (MAC):** Спречување на колизии преку техники како CSMA/CA;
- **Планирање на испраќање:** Врз основа на приоритет, енергија или временски интервали;

- **Обработка на загуби:** Реемитирање и потврдување на примени пакети.

Предизвици

- Ограничен опсег и пропусен опсег;
- Заштита од загуби, бучава и интерференции;
- Одложувања поради ниска фреквенција или sleep режими;
- Скалабилност и динамичко додавање на нови јазли.

9. Перформанси и функционалности

Оценувањето на оперативните системи за сензорски мрежи се базира врз нивните перформанси и поддржаните функционалности, бидејќи тие директно влијаат врз животниот век, сигурноста и корисноста на мрежата.

Перформансите и функционалностите на еден оперативен систем директно ја дефинираат неговата погодност за конкретни апликации, била да е тоа ниска енергија, реално време, безбедност или скалабилност. Изборот на ОС во WSN треба секогаш да биде балансиран според техниките ограничувања и очекуваните резултати.

Во многу апликации (здравство, војска, безбедност), реалновременската реакција е клучна. RTOS оперативните системи овозможуваат гарантирани времиња на одговор.

Добриот оперативен систем треба да биде лесно проширлив, т.е. да може да се користи во мали мрежи со неколку јазли, но и во големи мрежи со стотици уреди, без губење на стабилност или брзина. Contiki и RIOT се високо скалабилни, со модуларен дизајн кој овозможува вклучување или исклучување на функционалности по потреба.

Безбедноста во WSN е предизвик поради ограничената меморија и процесорска моќ, отворената безжична комуникација, како и поради ризикот од напади (eavesdropping, spoofing, DoS). Функции што се обезбедуваат се

шифрирање на податоци (AES, ECC), автентикација и контрола на пристап, како и безбедно рутирање и валидација на пакетите.

Потрошувачка на енергија

Потрошувачката на енергија е најважниот критериум при дизајнирањето на ОС за WSN. Сензорските јазли често се напојуваат со батерии и функционираат во тешко достапни средини, па оперативниот систем мора да го минимизира времето кога радио модулот е активен (Duty cycling), да менаџира задачи така што се избегнува непотребна активност и да користи енергетски-ефикасни протоколи (на пример, B-MAC, ContikiMAC).

На пример, TinyOS е дизајниран така што целиот систем се активира само кога е неопходно.

Поддршка за мултитаскинг

Различните оперативни системи поддржуваат различни модели на извршување на повеќе задачи и овие модели овозможуваат паралелна обработка на настани, мерења и комуникација:

- **TinyOS** користи кооперативен настански модел без вистински мултитаскинг – секоја задача мора брзо да заврши;
- **Contiki** користи protothreads, кои се полесна алтернатива на класичните нишки;
- **RIOT** нуди вистински мултитаскинг со приоритети, исто како класичните RTOS (Real-Time Operating Systems).

10. Процеси и сервиси

Во класичните оперативни системи, процесите претставуваат изолирани единици на извршување. Меѓутоа, во оперативните системи за безжични сензорски мрежи (WSN), концептот на процеси е модифициран и

прилагоден на ограничените ресурси, минималната енергија и специфичниот архитектурен модел на јазлите.

Управувањето со процеси и сервиси во оперативните системи за WSN е дизајнирано да биде минимално, флексибилно и ефикасно, при што се балансираат ограничувањата на хардверот со потребите на апликациите. Овие механизми овозможуваат реактивност, модуларност и скалабилност, клучни за успехот на секоја сензорска мрежа.

Модели на процеси

- **Настански модел (event-driven):** се користи во TinyOS – сите активности се иницирани од настани – прекини, мерења, пристигнати пораки. Наместо класични процеси, се користат мали задачи (tasks) кои се извршуваат по редослед. Овој модел троши минимална меморија и е многу енергетски ефикасен;
- **Нитки и протонишки (threads/protothreads):** Contiki користи protothreads, кои изгледаат како класични нишки но со многу помал трошок. RIOT и MANTIS овозможуваат вистински паралелни процеси (multithreading) со контрола на приоритети, тајмери и синхронизација.
- **Хибридни системи:** Некои оперативни системи, како на пример, Contiki комбинираат настани и процеси за да се добие баланс меѓу ефикасност и флексибилност.

Системски сервиси

Сервисите во WSN се вградени функционалности што оперативниот систем ги обезбедува на апликациските модули, како:

- **Тајмери и синхронизација:** За мерење време меѓу настани, повторливо извршување, закажување задачи. ОС обезбедуваат прецизни или груби тајмери во зависност од ресурсите;

- **Меѓупроцесна комуникација (IPC):** Механизми за комуникација помеѓу компоненти или процеси, на пример, преку настани, пораки, споделени бафери;
- **Управување со ресурси:** Пристап до сензори, радио, меморија, процесор – преку API со ограничени привилегии. Секој процес добива дел од ресурсите или пристапува до нив по барање;
- **Пристап до сензори и периферии:** ОС обезбедува слој за апстракција – не се пристапува директно до хардверот, туку преку софтверски сервиси;
- **Раководење со прекини:** Прекините се најважен механизам за реакција на надворешни настани. ОС мора брзо и ефикасно да го прекине тековното извршување и да премине на критичен код.

Управување со процеси

Некои ОС поддржуваат распоредување (scheduling) на процеси или задачи, базирано на приоритет, време на закажување и енергетска состојба на јазолот. Контролата на процесите вклучува:

- Создавање и уништување на процеси;
- Прекинување, паузирање и продолжување;
- Делба на ресурси (време на процесор, меморија, комуникациски капацитет).

1. Споредба со традиционални оперативни системи

КРИТЕРИУМ	ТРАДИЦИОНАЛЕН ОС	WSN ОС
Процеси	Тешки, изолирани	Лесни, често кооперативни
Распоредување	Комплексни алгоритми	Едноставни, FIFO или приоритет
IPC механизми	Мемориска споделба	Настани, сигнали, опашки
Тајмери	Прецизни, повеќениво	Едноставни, често груби

II. Анализа на конкретни оперативни системи за сензорски мрежи

11. TinyOS

TinyOS претставува еден од првите и најзначајни оперативни системи развиени специјално за безжични сензорски мрежи. Развиен е од Универзитетот во Беркли во рамките на проект кој има за цел да обезбеди лесна, енергетски ефикасна и компонентно-базирана софтверска платформа за уреди со ограничени ресурси. Тој е наменет за сензорски јазли со многу мала процесорска моќ, ограничена меморија и ограничен енергетски капацитет, како што се батерии кои не може лесно да се заменат или полнат.

Архитектурата на TinyOS е заснована на компоненти кои меѓусебно комуницираат преку строго дефинирани интерфејси. Сите потребни функционалности за една апликација се избираат во моментот на компилација при што се создава единствен бинарен фајл што се вчитува на уредот. Овој пристап овозможува брзо извршување, минимална употреба на меморија и предвидливост во перформансите, што е особено важно кај системи кои мораат да функционираат автономно подолг временски период.

Програмирањето на TinyOS се врши со помош на јазикот nesC, кој е дизајниран како продолжение на програмскиот јазик C, но со специфични механизми за работа со настански модели за управување со ресурсите на сензорските јазли. nesC овозможува дефинирање на компоненти, имплементација на интерфејси и повикување на настани преку декларативна синтакса. Иако ја подобрува сигурноста и ја намалува комплексноста на кодот на runtime, овој јазик има стрмна крива на учење и не е интуитивен за почетници.

Извршувањето во TinyOS се базира на настански модел. Наместо класичен мултитаскинг, сите активности се иницирани од настани – како што се пристигнување на податочни пакети, мерење со сензори или тајмерски прекини – и обработени со кратки задачи кои се извршуваат една по една.

Овој модел овозможува висока енергетска ефикасност и ја елиминира потребата од сложено управување со стекови или прекинување на процеси.

TinyOS користи статичко управување со меморијата, каде што сите структури и бафери се дефинираат при компилација. Иако ова ја ограничува флексибилноста, истовремено значително ја зголемува стабилноста и ја намалува потрошувачката на ресурси. Енергетската ефикасност е дополнително постигната преку автоматизирано контролирање на sleep и wake-up циклусите на сензорските јазли, при што радио модулите остануваат неактивни најголем дел од времето.

Во однос на мрежната комуникација, TinyOS поддржува основни комуникациски протоколи засновани на IEEE 802.15.4, како и сопствени имплементации на активни пораки, мултихоп рутирање и протоколи за собирање на податоци како што е Collection Tree Protocol (CTP). Иако не е дизајниран со поддршка за IPv6 или современи интернет протоколи, TinyOS останува солидна платформа за сценарија каде што се потребни лесни, автономни и долготрајни мрежни инсталации.

Примената на TinyOS е распространета во научни и истражувачки проекти како мониторинг на шумски пожари, следење на земјотреси, надзор на еколошки услови и разни експериментални WSN инсталации. Сепак, поради недостиг на флексибилност и комплексноста на програмскиот јазик, се повеќе се заменува со понови и пофлексибилни оперативни системи како Cintiki и RIOT.

TinyOS останува основоположник во областа на сензорските оперативни системи и е особено применлив во апликации каде што ниската потрошувачка, сигурноста и едноставноста се приоритет.

12. Contiki OS

Contiki е отворен, лесен оперативен систем развиен за употреба во вградени системи и безжични сензорски мрежи. Тој е првично развиен од

Адам Данкелс во Шведскиот институт за компјутерска наука, со цел да понуди баланс помеѓу минимален мемориски отпечаток и напредни мрежни функционалности, што го прави особено погоден за апликации во IoT (Internet of Things).

Она што го издвојува Contiki од другите слични системи е неговата флексибилна архитектура и можноста да работи на уреди со само неколку килобајти RAM и ROM, додека истовремено поддржува мрежни стекови како IPv6 и протоколи како 6LoWPAN и RPL. Тоа го прави еден од првите оперативни системи за сензорски мрежи кој овозможува директна поврзаност, без потреба од дополнителна хардверска поддршка или мрежни мостови.

Во поглед на моделот на извршување, Contiki користи хибриден пристап, комбинирајќи настански модел со лесни нишки наречени protothreads. Овие protothreads изгледаат како класични нишки, но всушност се кооперативни и не користат посебен стек, што значително ја намалува потрошувачката на меморија. Оваа архитектура овозможува развој на апликации со повеќе паралелни активности без сложеност карактеристична за вистински мултитаскинг.

Contiki обезбедува широк спектар на вградено мрежни функционалности, меѓу кои најзначајни се: uIP (микро TCP/IP стек), Rime (едноставен комуникациски протокол за WSN), како и 6LoWPAN стек кој овозможува IPv6 адресирање во средини со ниска пропусност. Благодарение на овие карактеристики, Contiki е меѓу првите избори за развој на паметни уреди и сензорски системи кои комуницираат преку интернет.

Иако Contiki работи на голем број хардверски платформи, најчесто се користи на микроконтролери од фамилиите MSP430, ARM и AVR. За олеснување на развојот и тестирањето, Contiki доаѓа со симулаторот Cooja, кој овозможува реална симулација на мрежно однесување, енергетска потрошувачка и комуникациски протоколи. Овој алат е широко користен во

истражувачката заедница, бидејќи овозможува симулација и анализа на големи мрежи пред нивно реално имплементирање.

Во однос на енергетската ефикасност, Contiki поддржува динамичко вклучување и исклучување на модулите, вклучувајќи duty cycling техники за радио модулите. Иако не е толку екстремно оптимизиран како TinyOS, Contiki обезбедува доволен баланс меѓу потрошувачка и функционалност, што го прави погоден за долгорочни апликации.

Програмскиот јазик што се користи е класичен C, што го прави полесен за учење и користење отколку nesC кај TinyOS. Тоа овозможува побрз развој на апликации и поголема достапност на ресурси, примери и заедница која активно придонесува во развојот на системот.

Contiki се применува во најразлични домени: од следење на животна средина и земјоделство, до индустриски апликации, паметни градови и автоматизација на домови. Неговата компатибилност со интернет протоколи, поддршката за симулација и модуларниот дизајн го прават особено атрактивен за академски истражувања и развој на IoT уреди.

Contiki претставува моќна и модерна платформа за сензорски мрежи, која успешно ги комбинира едноставноста, модуларноста и интернет поврзаноста. Тој е еден од најрелевантните оперативни системи за развој на интелигентни, поврзани сензорски системи во современиот дигитален свет.

13. RIOT OS

RIOT OS е модерен, отворен оперативен систем дизајниран за интернет поврзани вградени системи, со особен фокус на безжични сензорски мрежи и IoT уреди. Тој претставува реално-временски систем (RTOS) кој поддржува вистински мултитаскинг, сигурно управување со ресурси и стандардизирана мрежна комуникација. Развиен е од заедница на

академски и индустриски истражувачи и е достапен под LGPT лиценца, што овозможува широка примена и континуиран развој.

Она што RIOT го издвојува од другите оперативни системи е неговата компатибилност со POSIX и C стандардите, што значи дека голем дел од апликациите развиени за други платформи може лесно да се пренесат. Оваа одлика го прави RIOT особено погоден за развивачи кои веќе имаат искуство со Unix базирани системи. Дополнително, RIOT се програмира на C и C++, овозможувајќи богата поддршка за модуларност и објектно ориентиран пристап.

RIOT користи вистински мултитаскинг со нишки (threads), што значи дека секоја задача може да се извршува во сопствен контекст со посебен стек и приоритет. Распореѓувањето на процесите се врши со помош на прекинувања и тајмери, овозможувајќи брза реакција и поддршка за реалновременски апликации. Ваквиот модел е многу пофлексибилен од настанскиот модел кај TinyOS или protothreads кај Contiki, но бара и поголеми ресурси, особено меморија.

Во однос на мрежната комуникација, RIOT нуди широк опсег на поддржани протоколи, вклучувајќи IPv6, 6LoWPAN, RPL, CoAP и други стандардизирани протоколи за IoT. Тој поддржува мрежни стекови преку модуларни интерфејси, што овозможува лесна адаптација кон различни комуникациски технологии, како IEEE 802.15.4, BLE, LoRa и Wi-Fi. Благодарение на овие карактеристики, RIOT е совршено позициониран за развој на поврзани уреди во реални IoT мрежи.

Безбедноста е исто така важен елемент во RIOT. Оперативниот систем вклучува основни механизми за шифрирање, автентикација и сигурно рутирање, како и поддршка за интеграција на дополнителни криптографски библиотеки. Ова е клучно за индустриски и домашни апликации каде што доверливоста и заштитата на податоците се приоритет.

Мемориското управување кај RIOT е динамичко, со алокација на стекови и бафери при стартување на секој процес, што овозможува флексибилност и скалабилност. Иако тоа значи нешто поголема потрошувачка на ресурси, системот е оптимизиран така што може да работи и на уреди со 8 KB RAM и 32 KB ROM, што е соодветно за голем број микроконтролери.

RIOT се користи во различни полиња, вклучувајќи индустриска автоматизација, паметни градови, медицински уреди, системи за надзор и едукација. Има широка и активна заедница која постојано придонесува со нови модули, драјвери и примери. Покрај тоа, RIOT може да се извршува и како виртуелна инстанца на Linux (native port), што овозможува лесна симулација и тестирање без физички уреди.

RIOT е моќна и флексибилна платформа за развој на комплексни и поврзани апликации во IoT средини. Со поддршка за вистински мултитаскинг, стандардизирана мрежна комуникација и POSIX компатибилност, RIOT претставува одличен избор за проекти кои бараат сигурност, скалабилност и современи функционалности.

14. MANTIS OS

MANTIS (Multimodal system for NeTworks of In-situ wireless Sensors) е еден од раните оперативни системи развиен специјално за безжични сензорски мрежи, при универзитетот во Колорадо. Неговата основна цел е да обезбеди лесна, мултитаскинг околина која поддржува динамичко извршување на апликации и мрежни задачи на хардвер со ограничени ресурси. Она што го прави MANTIS посебен во споредба со другите оперативни системи од своето време е неговата поддршка за класичен multithreading, што е реткост во овој домен.

За разлика од TinyOS, кој се базира на настански модел и користи нестандартни програмски парадигми, MANTIS се програмира со класичен ANSI C и обезбедува API кој е лесно разбирлив за програмери со основно

познавање на програмски јазици. Системот нуди реална поддршка за нишки, при што секоја нишка има свој стек и приоритет, а управувањето со извршувањето се реализира преку вграден распоредувач на процеси. Оваа функционалност овозможува паралелно извршување на апликации и комуникациски задачи, што го прави MANTIS погоден за апликации со поголема сложеност.

Мемориското управување во MANTIS се заснова на динамичка алокација, каде што секоја нишка добива сопствен стек. Ова нуди флексибилност, но истовремено поставува повисоки барања во однос на контролата на потрошувачката и стабилноста на системот. Иако ова значи дека MANTIS не е толку енергетски ефикасен како TinyOS, тој овозможува поголема функционалност и подобра реакција на паралелни настани.

Во поглед на комуникацијата, MANTIS нуди поддршка за основни комуникациски стекови базирани на IEEE 802.15.4, со можност за имплементација на едноставни мрежни протоколи. Иако не нуди вградена поддршка за IPv6 или напредни IoT протоколи како 6LoWPAN, системот може да се проширува со сопствени решенија, благодарение на неговата модуларна архитектура.

MANTIS е дизајниран да биде пренослив на различен хардвер, вклучувајќи AVR и MSP430 микроконтролери, но и за Linux платформи, каде што може да се извршува за симулација или тестирање. Во практика, овој оперативен систем, најмногу се користи во истражувачки и академски проекти, каде што е потребна флексибилност при развојот и експериментирањето со нови алгоритми за рутирање, синхронизација и распоредување на задачи.

Иако во денешно време MANTIS не е толку популарен и не се развива активно како Contiki или RIOT, неговата концептуална структура претставува важен чекор во еволуцијата на оперативните системи за безжични сензорски мрежи, особено во правец на поддршка за класични програмски парадигми и развој на понапредни runtime системи.

15. LiteOS

LiteOS е оперативен систем со отворен код наменет за вградени системи и безжични сензорски мрежи, со особен фокус на IoT уреди. Тој е развиен од страна на Huawei, и претставува еден од ретките системи од оваа класа кој поддржува корисничко искуство слично на Unix, со командна линија, систем на датотеки и поддршка за динамичко вчитување на апликации.

Една од најважните карактеристики на LiteOS е тоа што обезбедува поддршка за вистински multithreading, при што секој процес или нишка може да работи со свој контекст, приоритет и време на извршување. Ова овозможува паралелно извршување на различни функционалности, како што се сензорски мерења, обработка на податоци и комуникација, без притоа да се загрозува стабилноста или перформансите на системот.

Програмирањето на LiteOS се врши во C, со стандардна структура на API кои потсетуваат на класичните RTOS решенија. За разлика од TinyOS, кој бара изучување на специфичен јазик (nesC), LiteOS е попристапен за почетници и инженери кои веќе имаат искуство со вградени системи и real-time програмирање. Системот поддржува динамичко вчитување и ажурирање на код, што овозможува флексибилно одржување на уредите во поле, без потреба од нивно физичко ресетирање или повторна инсталација.

Во поглед на мрежната поддршка, LiteOS обезбедува основни комуникациски модули, со можност за проширување преку интеграција на надворешни протоколи како IPv6 и 6LoWPAN. Иако не е првично фокусиран на мрежна комуникација како Contiki или RIOT, LiteOS сепак обезбедува доволна флексибилност за развој на IoT решенија во контролирани, затворени мрежни системи.

Архитектонски, LiteOS е модуларен и лесен, прилагоден за микроконтролери со ограничени ресурси. Тој вклучува основни сервиси како управување со процеси, тајмери, синхронизација и управување со енергија. Потрошувачката на енергија е оптимизирана преку поддршка за sleep

режими и duty cycling, што овозможува уредите да функционираат долго време без батерии.

LiteOS се користи пред се во индустриски проекти, IoT решенија на Huawei, и како основа за некои паметни уреди, посебно во Кина. Иако нема толку широка академска заедница како TinyOS или Contiki, неговата стабилност, лесна употреба и реално-временски способности го прават атрактивен избор за комерцијални апликации каде што е потребен сигурен embedded систем.

Во контекст на споредба со другите ОС за сензорски мрежи, LiteOS нуди релативно високо ниво на функционалност, но со нешто поголема потрошувачка на ресурси. Тоа го прави соодветен за уреди со умерени хардверски можности, кои бараат сигурна и флексибилна платформа за долг работен век.

16. Критична анализа

Со цел да се разбере соодветноста на различни оперативни системи за различни апликации во безжичните сензорски мрежи, неопходно е критички да се споредат врз основа на повеќе параметри: потрошувачка на енергија, поддршка за мрежни протоколи, модел на извршување, мемориско управување, безбедност и скалабилност.

2. Споредбена табела

ОС	Процесен	Меморија	Мрежна	Програмск	Поддршка	Безбеднос
----	----------	----------	--------	-----------	----------	-----------

	модел		поддршка	и јазиј	за IPv6	т
TinyOS	Event-driven	Статичка	IEEE 802.15.4	nesC	Ограничен а	Основна
Contiki	Protothreads	Динамичка	Rime, uIP, 6LoWPAN	C	Да	Средна
RIOT	Thread-based	Динамичка	6LoWPAN, RPL, CoAP	C/C++	Да	Напредна
LiteOS	Multithreading	Динамичка	Proprietary	C	Ограничен а	Средна
MANTIS	Multithreading	динамичка	IEEE 802.15.4	C	Не	Основна

- **TinyOS**

- ✓ Исклучително енергетски ефикасен.
- ✗ Ограничена поддршка за IP комуникација и комплициран програмирачки модел (nesC).

- **Contiki**

- ✓ Добар баланс помеѓу ефикасност и функционалност.
- ✓ IPv6 поддршка преку uIP стек.
- ✗ Protothreads се кооперативни – не поддржуваат вистински паралелизам.

- **RIOT**

- ✓ Напреден RTOS со вистински нишки и POSIX API.
- ✓ Висока модулартност и компатибилен со IoT.
- ✗ Поголема потрошувачка на ресурси во споредба со TinyOS.

- **LiteOS**

- ✓ Лесен за користење, сличен на Unix.
- ✗ Се уште експериментален и со ограничена заедница.

- **MANTIS**

- ✓ Добра поддршка за паралелно извршување.

- × Нема IP стек и слаба активност во заедницата.

Критички осврт

- **Перформанси:**
TinyOS има најниска потрошувачка, но RIOT е покомплетен во функционалност;
- **Скалабилност и модуларност:**
Contiki и RIOT нудат модуларен дизајн, што овозможува лесна адаптација;
- **Поддршка за IoT:**
RIOT и Contiki се најсоодветни за современи IoT апликации, благодарение на IPv6 и CoAP;
- **Програмирање и развој:**
RIOT и Contiki се полесни за учење и развој, додека nesC (TinyOS) има поголема крива на учење;
- **Безбедност:**
RIOT предничи со вградени безбедносни модули и поголема флексибилност.

3. Избор според апликација

Апликација	Препорачан ОС
Мониторинг со ниска потрошувачка	TinyOS
IoT уреди со интернет поврзување	RIOT, Contiki
Образовни и истражувачки цели	Contiki
Вистинско реално време	RIOT, LiteOS

Ниту еден оперативен систем не е “универзално најдобар“. Изборот треба да се базира на конкретните потреби на апликацијата, достапните ресурси на сензорските јазли и барањата за комуникација, енергија и сигурност. Во пракса, се применува компромис меѓу минимализмот и функционалноста.

17. Заклучок

Оперативните системи за сензорски мрежи претставуваат фундаментален дел од софтверската архитектура на секој интелигентен сензорски систем. Тие обезбедуваат основни механизми за управување со процеси, меморија, енергија, комуникација и хардверски ресурси, при што мора да бидат дизајнирани да функционираат во околина со екстремно ограничени ресурси.

Преку оваа семинарска работа се разгледани различните типови на оперативни системи, нивната архитектура, модели на процеси, комуникациски механизми, пристап до ресурси и нивните функционалности. Секој од нив има сопствени предности и недостатоци, кои го прават соодветен за специфични апликации.

На пример, TinyOS е одличен избор за апликации со ниска потрошувачка на енергија и едноставна логика, додека RIOT и Contiki со погодни за IoT апликации каде се потребни повеќе мрежни и безбедносни функционалности. Изборот на оперативен систем секогаш треба да биде заснован на баланс помеѓу перформанси, сигурност, енергетска ефикасност и програмска поддршка.

Како што технологија напреднува и како што се проширува примената на сензорските мрежи во нова област како паметни градови, паметно земјоделство и индустрија, потребата за флексибилни, сигурни и ефикасни оперативни системи ќе расте. Затоа идните истражувања и развој треба да се насочат кон адаптивни, самооптимизирачки системи кои ќе можат да одговорат на динамички услови во реално време.

18. Користена литература

- Karl, H., & Willig, A. (2005). *Protocols and Architectures for Wireless Sensor Networks*. Wiley-Interscience.
- Dunkels, A., Gronvall, B., & Voigt, T. (2004). *Contiki - A Lightweight and Flexible Operating System for Tiny Networked Sensors*. Proceedings of the First IEEE Workshop on Embedded Networked Sensors (EmNetS-I).
- Levis, P., Madden, S., Polastre, J., Szewczyk, R., Whitehouse, K., Woo, A., ... & Culler, D. (2005). *TinyOS: An Operating System for Sensor Networks*. Ambient Intelligence, Springer.
- Boulis, A. (2003). *Castalia: Revealing Pitfalls in Designing Distributed Algorithms in WSNs*. NICTA & University of New South Wales.
- Eswaran, A., Sankar, L., & Rajagopal, V. (2009). *Operating Systems for Wireless Sensor Networks: A Survey*. International Journal of Scientific & Engineering Research.